EventOps Intelligence

Technical Executive Brief

Assessment delivery | 10 October 2026 | Implementation and validation status

Purpose

EventOps Intelligence gives fictional event operations staff a single workspace for asking evidence-grounded questions, reconciling attendee records and understanding how an AI-assisted workflow reached its results. It combines a structured retrieval system with a controlled autonomous verification agent, backed by durable application state.

The product includes sign-in, an operational dashboard, Knowledge Assistant, Agent Workspace, Verification Results and Execution History. Staff can expand source evidence, inspect tool outcomes, record review notes and export only verified attendees. The visual design uses a restrained editorial layout, rounded controls and responsive navigation.

Engineering approach

The React/Vite/TypeScript frontend uses a FastAPI API with validated requests and responses. PostgreSQL stores sessions, document ingestion metadata, source records and embeddings, conversations, runs, execution events, verification evidence and workflow checkpoints.

Live RAG uses Gemini document/query embeddings and cosine similarity over whole structured records. Exact event identifiers constrain retrieval. Gemini produces structured answers; the backend validates actual citation identifiers and rejects invalid references. Conversation history is stored in PostgreSQL and restored after refresh or restart.

LangGraph owns planning, conditional source retrieval, extraction, deterministic verification and review decisions. The live planner uses Gemini function calls restricted to three read-only tools: Calendar, Gmail and the published directory. The official Google Gen AI SDK also performs schema-constrained email extraction. Direct adapters and protected n8n workflow exports provide the integration layer.

Verification outcomes

Business rules preserve source authority. The directory establishes identity, organization, active status and event eligibility. Email provides explicit registration evidence; Calendar supplies invitation evidence. A model cannot mark an unresolved conflict verified.

The controlled EVT-101 fixture contains seven attendee groups: one verified record, five records requiring review and one unverified directory mismatch. It includes duplicate registrations, missing emails, name and organization conflicts, an inactive participant and absent reference data. EVT-102 and EVT-103 each contain three consistent registrations. Verified-only CSV prevents unresolved rows from entering the operational export.

Evidence established

The backend suite passed 32 automated tests and the frontend suite passed nine tests. The frontend production build succeeded. The updated npm dependency installation reported zero known vulnerabilities. The real local HTTP smoke test exercised PostgreSQL-backed sign-in, ingestion, RAG contracts, saved conversations, idempotent run initiation, agent execution, verification, CSV and human review notes.

A separate backend restart check confirmed that the same session, conversation, run and seven verification records survived. The three n8n exports imported successfully into the running local n8n instance after version identifiers were included. Browser testing verified that the dashboard renders actual database counts and saved evidence; a local CORS issue was corrected during those checks.

Delivery boundary

The implementation and local fixture journey are working. The assessment's mandatory public RAG deployment and genuine Gemini/Google/n8n demonstrations remain outstanding because account credentials and hosting authorization were not supplied. The operator explicitly requested continued implementation with the access blockers documented.

Fixture mode is visibly labeled. Its deterministic embedding and generation doubles do not prove semantic retrieval quality or successful external calls. No successful live deployment, evaluation score or deployment URL is invented.

Next required actions

Supply a free Gemini API key, configure and authorize a dedicated fictional Google account, choose managed PostgreSQL with durable free storage, and authorize the Vercel/Render deployment. Publish the directory, configure n8n OAuth/header credentials, then execute the live verification checklist and RAG evaluation cases. Use actual run identifiers, screenshots and URLs when updating the submission evidence.

The repository contains reproducible local setup, deployment configurations, architecture diagrams, eight recorded architecture decisions, security boundaries, exported workflows, evaluation specifications and a 25-minute technical walkthrough. No paid resources or billing changes were created.